Introduzione all'apprendimento automatico

Nicolò Giuliani

Indice

1 Introduzione **3**
1.1 Di cosa si occupa il Machine Learning? . . . 3
1.1.1 Predizioni . . . 3
1.1.2 Classi di problemi di apprendimento . . . 4
1.2 Features . . . 5
1.3 Alberi di decisione . . . 7
1.3.1 Alberi di decisione: un esempio . . . 7
1.3.2 Costruzione Top-down induttiva . . . 8
1.3.3 Definizione entropia . . . 8
1.3.4 Teoria della Informazione (Shannon) . . . 9
1.3.5 Meglio A o B? . . . 10
1.3.6 Il caso continuo . . . 10
1.4 Overfitting . . . 10
1.4.1 Evitare overfitting sugli alberi di decisione . . . 11
1.4.2 Gini's impurity . . . 11
1.4.3 Foreste (Random Forests) . . . 13
1.5 Approccio probabilistico . . . 13
1.5.1 Variabile aleatoria . . . 13
1.5.2 Spazio campionario ed eventi . . . 13
1.5.3 Assiomi della teoria della probabilità . . . 14
1.5.4 Teoremi derivati . . . 14
1.5.5 Variabili aleatorie multivalore . . . 14
1.5.6 Probabilità condizionata . . . 14
1.5.7 Eventi indipendenti . . . 15
1.5.8 Regola di Bayes . . . 15
1.5.9 Posterior, likelihood, prior e marginale . . . 15
1.6 Distribuzione congiunta e classificatori Bayesiani . . . 16
1.6.1 La distribuzione congiunta . . . 16
1.6.2 Uso della distribuzione congiunta . . . 16
1.6.3 Inferenza con la distribuzione congiunta . . . 16
1.6.4 Naïve Bayes: riassunto . . . 17
1.6.5 Tecnica di apprendimento: stima dei parametri (MLE's) . . . 18
1.6.6 Esempio: Naïve Bayes su Play Tennis . . . 18
1.7 Modelli generativi . . . 19
1.7.1 Tecniche generative . . . 19
1.7.2 Limitazioni . . . 19
1.8 Maximum Likelihood Estimation (MLE) . . . 20
1.9 Classificazione documento . . . 21
1.9.1 Maximum likelihood estimates (MLE's) . . . 22

1.9.2 Log likelihood . . . 22
1.9.3 Prodotto scalare, correlation, Cosine similarity . . . 23
1.10 La natura lineare di Naïve Bayes (caso booleano) . . . 24
1.10.1 Classificazione lineare . . . 24
1.11 Gaussian Naïve Bayes . . . 24
1.12 Accuratezza, Precisione e Richiamo . . . 25
1.12.1 Parametri descrittivi del modello . . . 25
1.12.2 Maximum Likelihood Estimates . . . 26
1.13 Classificatori discriminativi . . . 26
1.13.1 Regressione Logistica . . . 26
1.13.2 La funzione logistica . . . 26
1.13.3 Training per regressione logistica (caso binario) . . . 27
1.13.4 Gradiente . . . 27
1.14 La tecnica del gradiente . . . 28
1.14.1 Approccio iterativo . . . 28
1.14.2 La superficie di errore . . . 28
1.14.3 Online vs Batch learning . . . 29
1.14.4 Gradiente per regressione logistica . . . 29
1.14.5 Il processo di apprendimento . . . 29
1.14.6 Osservazione sulla derivata della sigmoid . . . 30
1.15 Modelli generativi e discriminativi . . . 30
1.15.1 Generative vs discriminative models . . . 30
1.15.2 Modelli generativi e discriminativi (esempio) . . . 30
1.16 Esempio: classificazione di cifre MNIST con Regressione Logistica . . . 31
1.16.1 I metodi lineari sono potenti . . . 31
1.16.2 Il dataset MNIST . . . 31
1.17 Classificazione con regressione multinomiale . . . 31
1.17.1 Classificazione con regressione multinomiale . . . 31
1.17.2 Visualizzazione dei pesi . . . 32

2 Introduzione alle reti neurali 32
2.1 Il neurone artificiale . . . 33
2.2 Funzioni di attivazione . . . 33
2.3 Il neurone corticale . . . 34
2.3.1 Alcune cifre sul cervello umano . . . 34
2.3.2 Confronto con il neurone corticale . . . 34
2.3.3 Motivazioni del calcolo neurale . . . 35
2.4 Topologie di rete . . . 35
2.4.1 Reti feed-forward e ricorrenti . . . 35
2.4.2 Livelli (layers) . . . 35
2.4.3 Livelli principali: dense layer . . . 35
2.4.4 Livelli principali: convolutional layer . . . 36
2.5 Approfondimento sul DL . . . 36
2.5.1 Capire il DL . . . 36
2.5.2 Framework per il DL . . . 37
2.5.3 Cenni storici . . . 37
2.5.4 Il lascito (legacy) . . . 37
2.5.5 La rivoluzione del Deep Learning . . . 37
2.5.6 La situazione a inizio secolo . . . 37
2.5.7 L'era del deep learning . . . 38

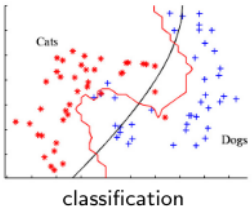
1 Introduzione

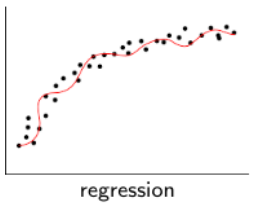
1.1 Di cosa si occupa il Machine Learning?

Si occupa di risolvere problemi complessi tramite tecniche algoritmiche specifiche. L'approccio generale del ML è

- definire una classe di modelli del problema su insieme di **parametri** Θ
- definire una metrica di valutazione, ovvero una misura dell'**errore** dei modelli
- modificare i parametri Θ per minimizzare l'errore sui **dati di training**

Quindi possiamo dire che `il machine learning è un procedimento di ottimizzazione`.





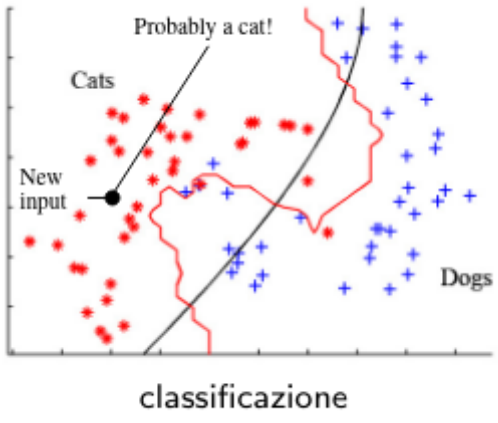
Il ML tratta di problemi di **ottimizzazione**!

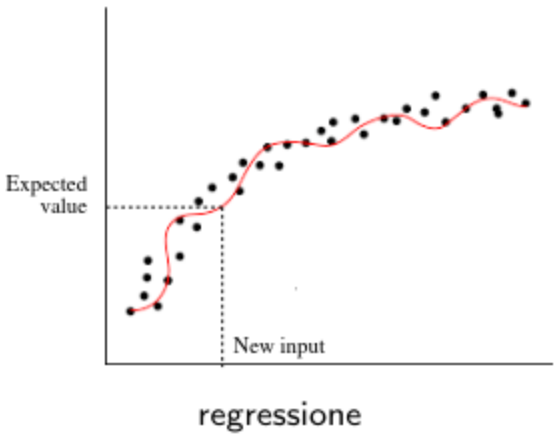
- Il `training set` fa sì che il tuning dei parametri è dato dalla esperienza passata, quindi una sorta di **apprendimento**.
- La `soluzione` spesso non esiste una soluzione in forma chiusa

Utilizziamo **tecniche iterative** per approssimare iterativamente la soluzione.

1.1.1 Predizioni

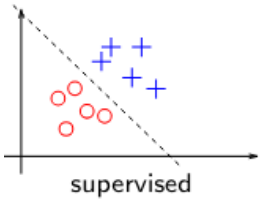
Attraverso modelli di ML possiamo anche stabilire correlazioni, clusters, ...



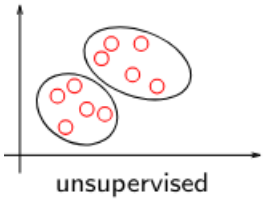


1.1.2 Classi di problemi di apprendimento

- Approfondimento supervisionato: inputs e outputs (etichette)
  - classificazione
  - regressione



- Approfondimento non supervisionato: inputs
  - clustering
  - analisi delle componenti
  - autoencoding



- Approfondimento con rinforzo
  - apprendere comportamenti
  - "model free" planning

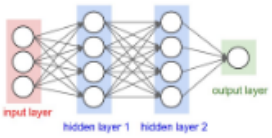


Molte tecniche differenti

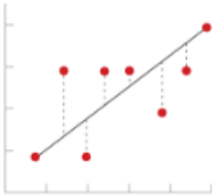
- Modi diversi per definire i modelli
  - alberi di decisione
  - modelli lineari
  - reti neurali
- Differenti funzioni di loss
  - errore quadratico
  - logistic loss
  - cross entropy
  - similarità del coseno
  - margine massimo



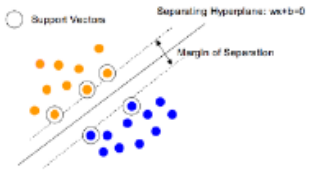
alberi di decisione



reti neurali



errore quadratico medio



margine massimo

1.2 Features

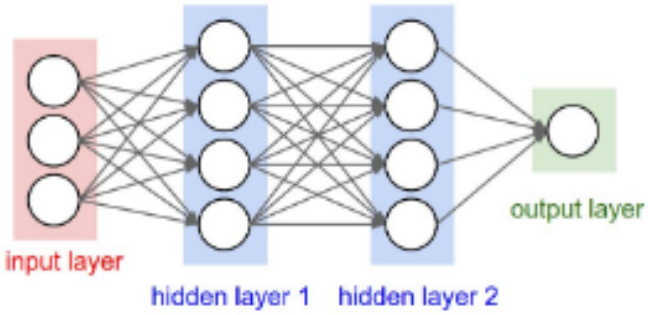
Ogni informazione realativa a un dato che ne descrive una sua proprietà è feature (caratteristica).

- Le features sono gli input del processo di apprendimento.
- L'apprendimento è molto sensibile alla scelta delle features.

Determinare delle buone caratteristiche è complesso.

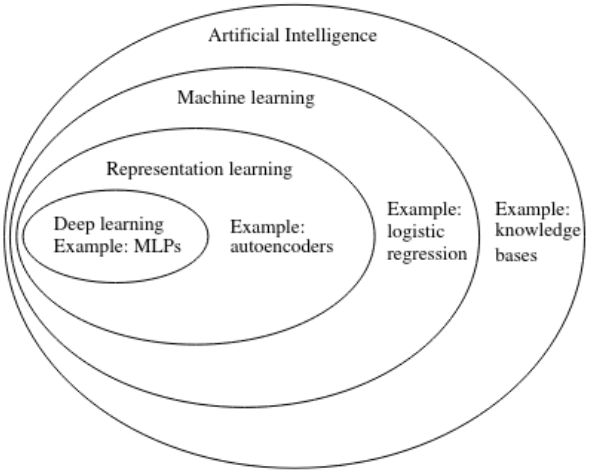
- **Approccio tradizionale**: determinare durante il preprocessing delle buone features, e applicare un metodo semplice ma robusto di apprendimento, ad esempio qualche tecnica lineare.
- **Approccio "deep"**: fornire in input dati grezzi e delegare alla macchina il compito di sintetizzare nuove features (rappresentatione interna).

`Deep learning` è implementato attraverso reti neurali profonde.
La rete neurale è profonda quando si hanno layers multipli:



Ogni layer sintetizza nuove features in funzione delle precedenti.

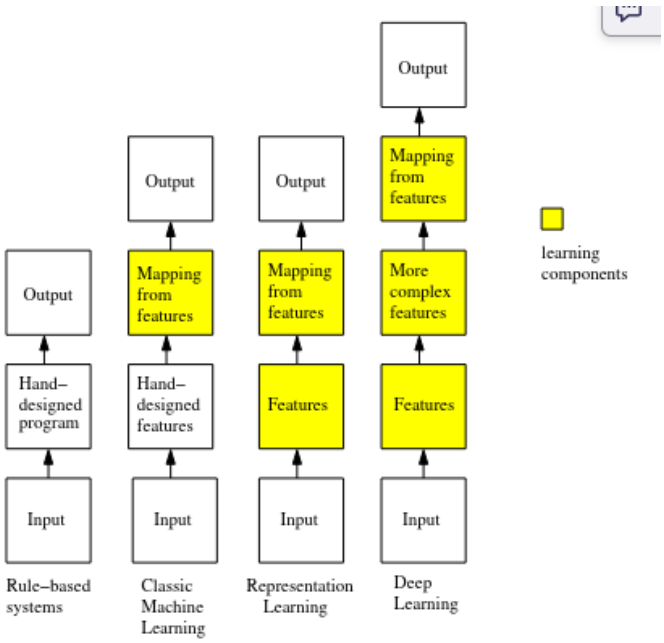
Relazione tra le aree di ricerca



L'AI, machine learning, deep learning:

- **Sistemi basati su conoscenza**: prendete un esperto, chiedetegli come risolverebbe il problema e poi provate a mimare il suo comportamento mediante regole logiche
- **Machine-Learning tradizionale**: prendete un esperto, chiedetegli di quali features avrebbe bisogno per risolvere un dato problema, raccogliete un insieme di osservazioni e lasciate alla macchina il compito di apprendere l'associazione
- **Deep-Learning**: liberatevi dell'esperto

Componenti addestrati mediante apprendimento



1.3 Alberi di decisione

Con gli albri di decisione possiamo esprimere qualsiasi funzione di classificazione.
Definiamo **traning set** un insieme di `esempi di allenamento`

$$\langle x^{(i)}, y^{(i)} \rangle$$

con

- $x^{(i)} \in X$ insieme degli inputs (vettore di feature) (*le feature possono essere sia discrete/continue*)
- $y^{(i)} \in Y$ insieme delgi outputs, **groudn truth**
- i indice dell'istanza di training

Definiamo **problema** trovare la funzione che "mappa" $x^{(i)}$ a $y^{(i)}$

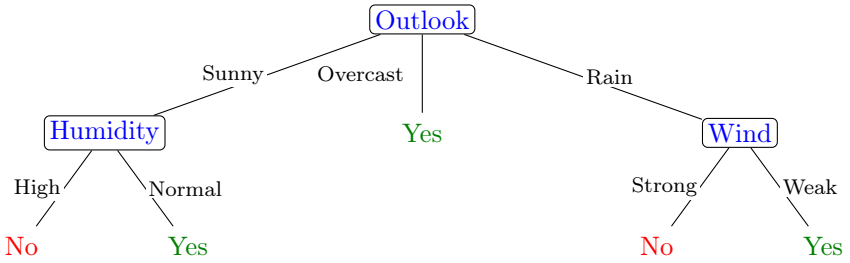
- Y discreto: problema di classificazione (predizione di una classe)
- Y continuo: problema di regressione (predizione di un valore)

Le tecniche di Machine Learning richiedono la scelta preliminare di uno `spazio di funzioni` H, all'interno del quale cercare la soluzione che `meglio approssima` i dati di training.
Un modello è un modo di specificare e calcolare una funzione nello spazio delle ipotesi H.

1.3.1 Alberi di decisione: un esempio

Ricorda: altezza albero $\leq$ numero features. Quindi il costo della predizione e' $O(altezza)$, ovvero molto basso.
È un buon giorno per giocare a tennis?

$$H : \text{Outlook} \times \text{Humidity} \times \text{Wind} \times \text{Temp} \rightarrow \text{Play Tennis?}$$



Ogni nodo testa un attributo (feature) X
Ogni arco uscente da un nodo corrisponde a uno dei possibili valori discreti di X
Ogni foglia predice la riposta Y (o un probabilità $P(Y|X)$)

- Insieme di input X:
  ogni istanza $x \in X$ è un vettore di features del seguente tipo:
  $\langle$ Humidity=high, Wind=weak, Outlook=rain, Temp=hot $\rangle$
- Funzione target $f : X \rightarrow Y$
  Y assume valori discreti (booleani)
- Spazio delle ipotesi $H = \{h \mid h : X \rightarrow Y\}$ (nessuna restrizione)
  - vogliamo modellare $h \in H$ con un albero di decisione
  - ogni istanza $x \in X$ definisce un cammino nell'albero che conduce a una foglia etichettata con $y \in Y$, corrispondente alla soluzione predetta

Outlook	Temp	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

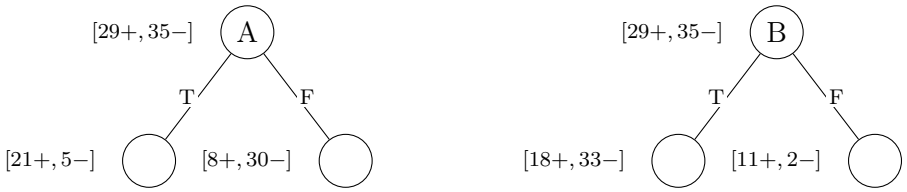
Expressività del modello Definiamo $X_1 \times X_2 \times \cdots \times X_n$ con $X_i = \{\text{true, false}\}$
Possiamo esprimere $Y = X_2 \vee X_5$?

1.3.2 Costruzione Top-down induttiva

Main loop:

1. assegnare al nodo corrente il "miglior" attributo X_i
2. creare un nodo figlio per ogni possibile valore di X_i e propagare i dati verso i figli a seconda del loro valore
3. per ogni nodo figlio, se tutti i dati del training set associati al nodo hanno una stessa etichetta y, marcare il nodo come foglia con etichetta y, altrimenti ripetere dal primo punto

Ma quale è il "miglior" attributo?



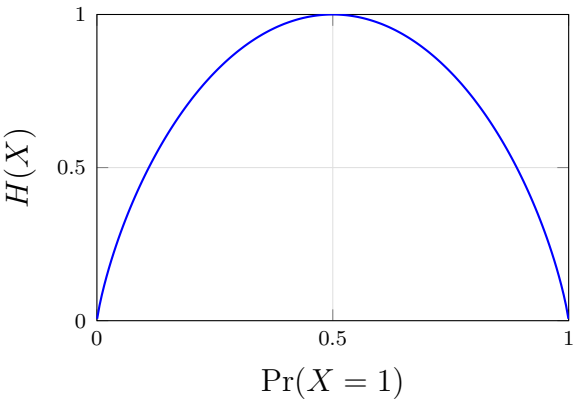
1.3.3 Definizione entropia

Definiamo **entropia** di una variabile aleatoria X

$$H(X) = -\sum_{i=1}^{n} P(X = i) \log_2 P(X = i)$$

dove n è il numero dei possibili valori di n.

L'entropia misura il grado di impurità (disordine) della informazione. È massima ($\log n$) quando X è uniformemente distribuita tra tutti i suoi n valori, e minima (0) quando è concentrata su un singolo valore.
Ricorda: una bassa entropia non misura se e' equamente distribuita ma focalizza le classi piu' frequenti.



1.3.4 Teoria della Informazione (Shannon)

L'Entropia è la quantità media di informazione prodotta da una sorgente stocastica di dati.

- Un evento con entropia 1 non trasmette Informazioni

$$I(1) = 0$$

- Dati due evento indipendenti con probabilità p_1, p_2 la probabilità congiunta è $p_1 p_2$ ma l'informazione acquisita è la somma delle informazioni dei due eventi indipendenti

$$I(p_1 p_2) = I(p_1) + I(p_2)$$

- Ci aspettiamo che l'informazione sia antimonotona rispetto alla probabilità ed è quindi naturale definire

$$I(p) = -\log(p)$$

Teoria dei codici (Shannon)

L'Entropia misura il numero medio di bits richiesti per trasmettere il valore prodotto da una sorgente stocastica X.
Se gli eventi non hanno la stessa probabilità possiamo migliorare la codifica!!

Entropia di X

$$H(X) = -\sum_{i=1}^{n} P(X = i) \log_2 P(X = i)$$

Entropia Condizionale di X dato uno specifico $Y = v$

$$H(X|Y = v) = -\sum_{i=1}^{n} P(X = i|Y = v) \log_2 P(X = i|Y = v)$$

Entropia Condizionale di X dato Y (media ponderata sugli m possibili valori di Y)

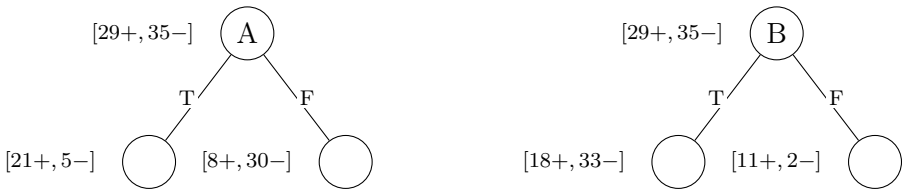
$$H(X|Y) = \sum_{v=1}^{m} P(Y = v) H(X|Y = v)$$

Guadagno Informativo tra X e Y

$$I(X,Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

1.3.5 Meglio A o B?

Misuriamo la riduzione di entropia della variabile target Y conseguente alla conoscenza di un qualche attributo X, cioè il guadagno informativo $I(Y,X)$ tra Y e X.



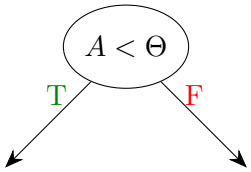
$$\begin{aligned}
H(Y) &= -(29/64)\cdot\log_2(29/64) - (35/64)\cdot\log_2(35/64) = .994\\
H(Y|A=T) &= -(21/26)\cdot\log_2(21/26) - (5/26)\cdot\log_2(5/26) = .706\\
H(Y|A=F) &= -(8/38)\cdot\log_2(8/38) - (30/38)\cdot\log_2(30/38) = .742\\
H(Y|A) &= .706\cdot 26/64 + .742\cdot 38/64 = .726\\
I(Y,A) &= H(Y) - H(Y|A) = .994 - .726 = .288
\end{aligned}$$

Per B otteniamo $H(Y|B) = .872$ e $I(Y,B) = .122$

Quindi, A è migliore!

1.3.6 Il caso continuo

Quando gli attibuti sono continui, prendiamo le decisioni in base a opportune soglie (thresholds):



- confrontiamo le varie soglie in base al loro guadagno informativo
- ma come scegliere le soglie candidate?
  - sampling a intervalli discreti prefissati
  - ordinare il train set rispetto a un dato attributo e scegliere come soglie i valori medi tra dati consecutivi

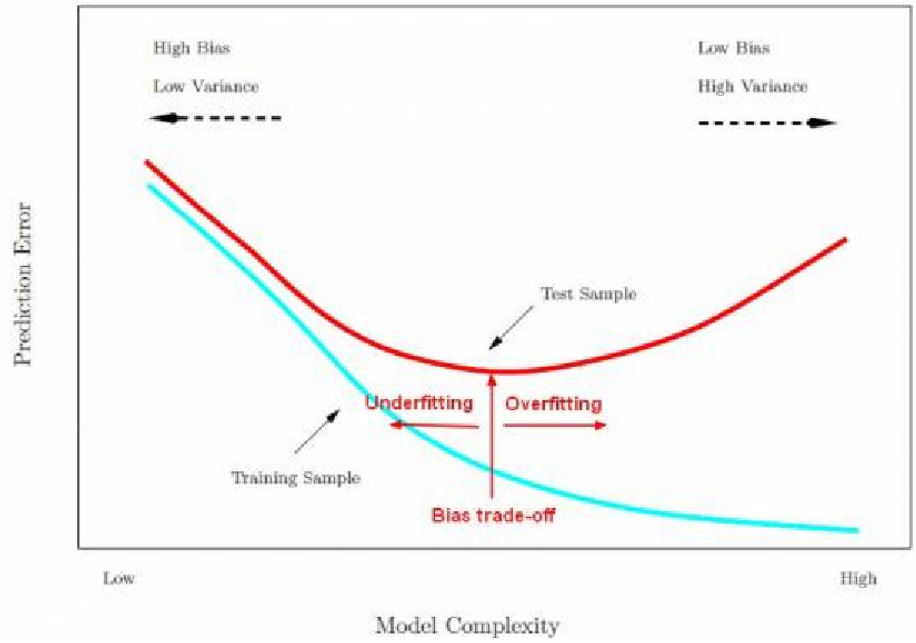
1.4 Overfitting

Consideriamo l'errore relativo all'ipotesi $h \in H$

- sul training set: $\text{error}_{train}(h)$
- sul data set (globale): $\text{error}_{\mathcal{D}}(h)$

Definiamo `overfitting` quando esiste un'ipotesi $h' \in H$ tale che

$$\text{error}_{train}(h') < \text{error}_{train}(h) \quad \text{ma} \quad \text{error}_{\mathcal{D}}(h') > \text{error}_{\mathcal{D}}(h)$$



Il problema? Non conosciamo $\mathcal{D}$! Quindi semplicement eprendiamo i dati a noi disposizione e li dividiamo in:

- training set: per costruire l'ipotesi h
- test set: utilizzato per misurare accuracy e overfitting h

Tipicamente si riserva una ulteriore porzione, il **test set**, per la valutazione finale.

1.4.1 Evitare overfitting sugli alberi di decisione

- `early stopping`: terminare la costruzione dell'albero non appena il miglioramento del modello non è più statisticamente significativo. Ad esempio, quando il guadagno informativo è inferiore ad una certa soglia, o il numero dei dati relativo al nodo è troppo piccolo.
- `post-pruning`: si sviluppa l'intero albero e quindi si procede a potarlo all'indietro.
  - post-pruning finalizzato alla riduzione dell'errore: costruire un albero di decisione completo per il training set, quindi ripetere la seguente operazione finché ogni ulteriore pruning non migliora l'accuratezza della predizione:
    1. per ogni sottoalbero, valutare (sul validation set) l'impatto della sua rimozione sulla accuratezza della classificazione
    2. effettuare (in modo greedy) il pruning del sottoalbero che ottimizza l'accuracy
  - nuovi dati di training
- acquisizione di nuovi dati di training

1.4.2 Gini's impurity

Gini's impurity (coefficiente di Gini) misura la probabilità che un generico elemento sia mal classificato in base alla classificazione corrente (è una possibile alternativa alla nozione di entropia).
Date m categorie, sia f_i la frazione dei dati con etichetta i. Questa è pari alla probabilità che un qualche input appartenga alla categoria i. La probabilità che venga classificato in modo errato è dunque $1 - f_i$, e il coefficiente di Gini è semplicemente la media pesata di questa quantità su

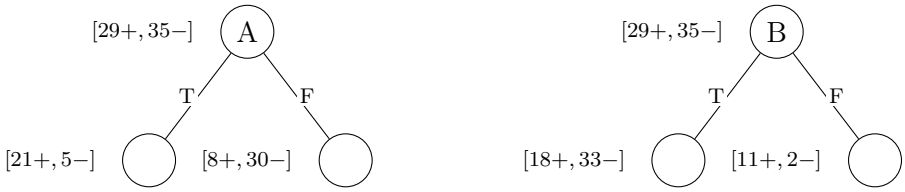
tutte le possibili categorie, cioè

$$\begin{aligned} I_G(f) &= \sum_{i=1}^{m} f_i(1-f_i) \\ &= \sum_{i=1}^{m} \left(f_i - f_i^2\right) \\ &= \sum_{i=1}^{m} f_i - \sum_{i=1}^{m} f_i^2 \\ &= \boxed{1 - \sum_{i=1}^{m} f_i^2} \end{aligned}$$

La metrica è applicata su ogni nodo figlio, e i valori sono sommati in modo pesato, in modo del tutto analogo a quanto avviene per l'entropia, al fine di ottenere una misura della qualità dell'attributo.

Gini: Meglio A o B?

Valutiamo gli split utilizzando il coefficiente di Gini.



$$\begin{aligned} I_G(A=T) &= 1 - (21/26)^2 - (5/26)^2 = .310 \\ I_G(A=F) &= 1 - (8/38)^2 - (30/38)^2 = .332 \\ I_G(A) &= .310 \cdot 26/64 + .332 \cdot 38/64 = .323 \end{aligned}$$

$$\begin{aligned} I_G(B=T) &= 1 - (18/51)^2 - (33/51)^2 = .456 \\ I_G(B=F) &= 1 - (11/13)^2 - (2/13)^2 = .260 \\ I_G(B) &= .456 \cdot 51/64 + .260 \cdot 13/64 = .416 \end{aligned}$$

Quindi, anche con Gini, A è migliore!

Vantaggi e svantaggi degli alberi di decisione

Aspetti positivi

- facili da capire: semplici regole logiche, gli alberi possono essere visualizzati
- poco preprocessing è di solito necessario (attenzione però al bilanciamento delle classi)
- costo predittivo molto basso
- può essere utilizzato sia con features discrete che continue

Aspetti negativi

- rischio elevato di overfitting
- selezione degli attributi piuttosto instabile al variare del dataset
- facile costruire alberi profondamente sbilanciati, specialmente in presenza di una classe dominante

1.4.3 Foreste (Random Forests)

Gli alberi di decisione sono abitualmente utilizzati come componenti delle cosiddette Random Forests, dove sono utilizzati con una tecnica ad **ensemble**.
Le tecniche ad "ensemble" sfruttano il principio che un **gran numero** di modelli **sufficientemente scorrelati** che operano come un comitato forniscono di solito risultati predittivi migliori dei singoli componenti.

Garantire la differenziazione degli alberi

- **Bagging**: allenare i modelli su sottoinsiemi random (bags) dei dati di input
- **Feature Randomness**: costruire gli alberi a partire da sottoinsiemi random delle features

1.5 Approccio probabilistico

Siamo interessati ad approssimare funzioni. Invece di calcolare

$$f : X \to Y$$

possiamo calcolare

$$p : X \to P(Y|X)$$

1.5.1 Variabile aleatoria

Una **variabile aleatoria** X denota l'esito di un fenomeno riguardo al quale sussiste incertezza, come ad esempio il risultato di un processo stocastico. Ad esempio

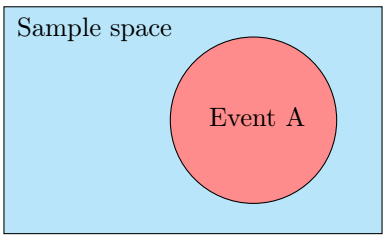
- $X =$ true se uno studente scelto a caso in aula è biondo
- $X =$ nome dello studente
- $X =$ true se due studenti scelti a caso sono nati lo stesso giorno dell'anno

Definiamo $P(X)$ come la frazione di volte in cui X è vero, in esecuzioni ripetute dell'esperimento.

1.5.2 Spazio campionario ed eventi

- l'insieme Ω dei possibili esiti di un esperimento casuale è detto **spazio campionario** (sample space)
- una variabile aleatoria è una funzione misurabile su Ω: discreta (es. $\Omega \to \{m, f\}$) o continua (es. $\Omega \to \mathbb{R}$)
- un **evento** è un qualche sottoinsieme di Ω; siamo interessati alla probabilità di eventi specifici, anche condizionati da altri eventi

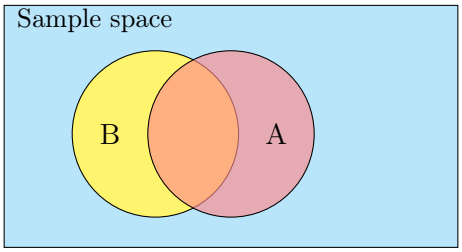
La scelta dello spazio campionario richiede cautela: spesso l'intuizione presuppone (erroneamente) che gli eventi elementari abbiano tutti la stessa probabilità.
Esempio: nel lancio di due dadi il risultato è un numero tra 2 e 12, ma questi valori non hanno tutti la stessa probabilità; conviene invece considerare come spazio campionario tutte le coppie dei valori dei due dadi, e definire X come la somma dei due risultati.



La probabilità $P(A)$ dell'evento A è il rapporto tra l'area di A e l'area dell'intero spazio campionario (supposto uniforme).

1.5.3 Assiomi della teoria della probabilità

$$0 \leq P(A) \leq 1$$
$$P(\text{True}) = 1 \qquad P(\text{False}) = 0$$
$$P(A \vee B) = P(A) + P(B) - P(A \wedge B)$$



1.5.4 Teoremi derivati

Dimostrazioni omesse.

$$\boxed{P(\neg A) = 1 - P(A)}$$

$$\boxed{P(A) = P(A \wedge B) + P(A \wedge \neg B)}$$

1.5.5 Variabili aleatorie multivalore

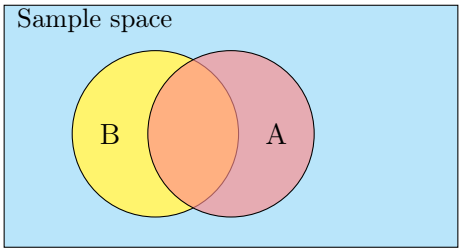
A è una variabile aleatoria discreta a k valori se può assumere esattamente uno dei valori $\{\nu_1, \nu_2, \ldots, \nu_k\}$.

$$P(A = \nu_i \wedge A = \nu_j) = 0 \quad \text{se } i \neq j$$
$$P(A = \nu_1 \vee A = \nu_2 \vee \cdots \vee A = \nu_k) = 1$$

1.5.6 Probabilità condizionata

La probabilità condizionata dell'evento A dato l'evento B è definita come

$$P(A|B) = \frac{P(A \wedge B)}{P(B)}$$



Corollario (chain rule):

$$P(A \wedge B) = P(B) \cdot P(A|B) = P(A) \cdot P(B|A)$$

1.5.7 Eventi indipendenti

Due eventi A e B sono detti **indipendenti** se

$$P(A|B) = P(A)$$

ovvero l'evento B non ha influenza sull'evento A. Come corollario

$$P(A \wedge B) = P(A) \cdot P(B)$$

e, siccome $P(A \wedge B) = P(B|A) \cdot P(A)$, abbiamo anche $P(B|A) = P(B)$: cioè anche B è indipendente da A.

1.5.8 Regola di Bayes

$$P(A|B) = \frac{P(A) \cdot P(B|A)}{P(B)}$$

1.5.9 Posterior, likelihood, prior e marginale

In generale, per ogni i, j

$$P(Y = y_i|X = x_j) = \frac{P(Y = y_i) \cdot P(X = x_j|Y = y_i)}{\sum_i P(Y = y_i) \cdot P(X = x_j|Y = y_i)}$$

dove il denominatore si ottiene marginalizzando (sommando su Y) $P(X = x_j) = \sum_i P(X = x_j, Y = y_i)$. In notazione compatta

$$\underbrace{P(Y|X)}_{\text{posterior}} = \frac{\overbrace{P(X|Y)}^{\text{likelihood}} \cdot \overbrace{P(Y)}^{\text{prior}}}{\underbrace{P(X)}_{\text{marginal likelihood}}}$$

"Marginale" in quanto stiamo marginalizzando (sommando/integrando) su Y.

1.6 Distribuzione congiunta e classificatori Bayesiani

1.6.1 La distribuzione congiunta

1. costruire una tabella con tutte le possibili combinazioni dei valori delle features
2. stimare la probabilità per ogni combinazione di valori

gender	working hours	health	prob (params)
F	≤ 40	poor	0.25
F	≤ 40	rich	0.03
F	> 40	poor	0.04
F	> 40	rich	0.01
M	≤ 40	poor	0.33
M	≤ 40	rich	0.10
M	> 40	poor	0.13
M	> 40	rich	0.11

Date n features booleane, dobbiamo stimare $2^n - 1$ parametri.

1.6.2 Uso della distribuzione congiunta

Ricorda: ricaviamo sia marginale/condizionata. Il costo e' direttamente proporzionale al numero di features.
Disponendo della distribuzione congiunta possiamo stimare la probabilità di **qualunque evento** esprimibile come combinazione logica delle features.

$$\underbrace{P(E) = \sum_{\text{row} \in E} P(\text{row})}_{\text{prob. marginale}}$$

Esempio: calcoliamo $P(M, \text{poor})$ dalla tabella precedente

$$\begin{aligned} P(M, \text{poor}) &= P(M, \text{poor}, \leq 40) + P(M, \text{poor}, > 40) \\ &= 0.33 + 0.13 = 0.46 \end{aligned}$$

1.6.3 Inferenza con la distribuzione congiunta

È altrettanto semplice calcolare la probabilità condizionata di un evento E_1 dato un altro evento E_2

$$P(E_1|E_2) = \frac{P(E_1 \wedge E_2)}{P(E_2)} = \underbrace{\frac{\sum_{\text{row} \in E_1 \wedge E_2} P(\text{row})}{\sum_{\text{row} \in E_2} P(\text{row})}}_{\text{prob. condizionata}}$$

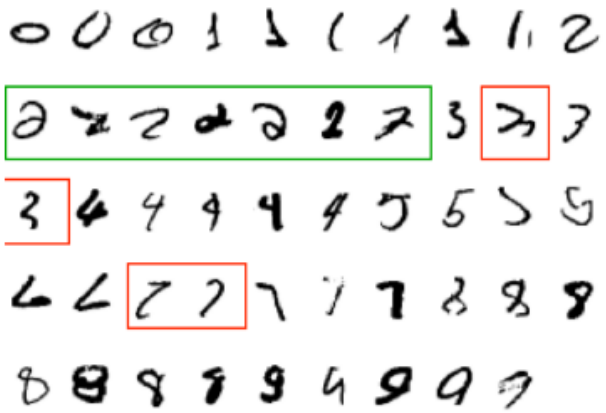
Riprendiamo l'esempio precedente e calcoliamo $P(\text{M}|\text{poor})$

$$P(\text{M}|\text{poor}) = \frac{P(\text{M}, \text{poor})}{P(\text{poor})} = \frac{0.46}{0.75} = 0.61$$

Probabilità condizionata vs. learning

Abbiamo appena visto che la conoscenza della distribuzione congiunta permette di calcolare la probabilità condizionata. Fine della storia?

Esempio: il caso di MNIST



- immagini di 28×28 pixels in scala di grigi che rappresentano esempi di cifre da 0 a 9 scritte a mano
- 60.000 immagini di training più altre 10.000 immagini di testing

Nel caso delle immagini ogni pixel è una feature, quindi una immagine di 28×28 pixels è rappresentata da un vettore di 784 features.
$\Rightarrow$ Abbiamo molte più righe che esempi di training. Un sacco di configurazioni non sono coperte dal training set.

Indipendenza condizionale

Due eventi X_i e X_j sono **indipendenti dato** Y se

$$P(X_i|X_j, Y) = P(X_i|Y)$$

Naïve Bayes

Per la **chain rule**

$$P(X_1, X_2|Y) = P(X_1|X_2, Y) \cdot P(X_2|Y)$$

e per l'**indipendenza condizionale**

$$P(X_1, X_2|Y) = P(X_1|Y) \cdot P(X_2|Y)$$

In generale

$$P(X_1, X_2, \ldots, X_n|Y) = \prod_i P(X_i|Y)$$

Quanti parametri per $P(X_1, \ldots, X_n|Y)$ (nel caso booleano)?

- senza indipendenza condizionale
- per Naïve Bayes

1.6.4 Naïve Bayes: riassunto

Ricorda: assume che le features siano indipendenti.
Bayes rule

$$P(Y = y_i|X_1, \ldots, X_n) = \frac{P(Y = y_i) \cdot P(X_1, \ldots, X_n|Y = y_i)}{P(X_1, \ldots, X_n)}$$

Naïve Bayes

$$P(Y = y_i|X_1, \ldots, X_n) = \frac{P(Y = y_i) \cdot \prod_j P(X_j|Y = y_i)}{P(X_1, \ldots, X_n)}$$

Classificazione di un nuovo dato $x_{\text{new}} = \langle x_1, \ldots, x_n \rangle$

$$Y_{\text{new}} = \arg\max_{y_i} \; P(Y = y_i) \cdot \prod_j P(X_j = x_j|Y = y_i)$$

1.6.5 Tecnica di apprendimento: stima dei parametri (MLE's)

Variabili aleatorie discrete X_i, Y.
Training (*maximum likelihood estimates*): per tutti i valori y_k di Y, stimare

$$\pi_k = P(Y = y_k) = \frac{\#_D\{Y = y_k\}}{|D|}$$

per ogni possibile valore x_{ij} di X_i, stimare

$$\theta_{ijk} = P(X_i = x_{ij} \mid Y = y_k) = \frac{\#_D\{X_i = x_{ij} \wedge Y = y_k\}}{\#_D\{Y = y_k\}}$$

dove $\#_D\{\cdot\}$ indica il numero di istanze del dataset D che soddisfano la condizione, e $|D|$ è la dimensione del dataset.

Classificazione di $a_{\text{new}} = \langle a_1, \ldots, a_n \rangle$

$$\begin{aligned} Y_{\text{new}} &= \arg\max_{y_k} \; P(Y = y_k) \cdot \prod_i P(X_i = a_i|Y = y_k) \\ &= \arg\max_{k} \; \pi_k \cdot \prod_i \theta_{ijk} \end{aligned}$$

supponendo che $a_i = x_{ij}$, cioè che a_i sia il j-esimo tra i possibili valori discreti dell'attributo X_i.

1.6.6 Esempio: Naïve Bayes su Play Tennis

Riprendiamo il training set visto per gli alberi di decisione (Outlook, Temp, Humidity, Wind $\rightarrow$ Play).

Outlook	Temp	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Stima dei parametri

$$\pi_{\text{yes}} = 9/14 \qquad \pi_{\text{no}} = 5/14$$

	$Y = \text{yes}$	$Y = \text{no}$
θ_{sunny}	2/9	3/5
θ_{cool}	3/9	1/5
θ_{high}	3/9	4/5
θ_{strong}	3/9	3/5

Predizione Nuova istanza: Outlook=Sunny, Temp=Cool, Humidity=High, Wind=Strong. Dobbiamo calcolare

$$\arg\max_{y \in \{\text{yes,no}\}} p(y) \cdot p(\text{sunny}|y) \cdot p(\text{cool}|y) \cdot p(\text{high}|y) \cdot p(\text{strong}|y)$$

ovvero confrontare

$$\pi_{\text{yes}} \cdot \theta_{\text{sunny,yes}} \cdot \theta_{\text{cool,yes}} \cdot \theta_{\text{high,yes}} \cdot \theta_{\text{strong,yes}} = \tfrac{9}{14} \cdot \tfrac{2}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} = .0053$$
$$\pi_{\text{no}} \cdot \theta_{\text{sunny,no}} \cdot \theta_{\text{cool,no}} \cdot \theta_{\text{high,no}} \cdot \theta_{\text{strong,no}} = \tfrac{5}{14} \cdot \tfrac{3}{5} \cdot \tfrac{1}{5} \cdot \tfrac{4}{5} \cdot \tfrac{3}{5} = .0205$$

No!

1.7 Modelli generativi

1.7.1 Tecniche generative

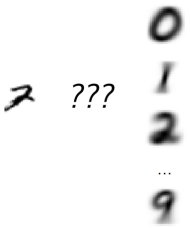
Ci interessa calcolare

$$P(Y = y_i \mid X_1, \ldots, X_n)$$

La regola di Bayes ci permette di invertire il problema in **data la categoria calcoliamo la distribuzione dei dati**.

Classificazione di un nuovo dato Classifichiamo un nuovo dato chiedendoci quale delle distribuzione input è quella che si avvicina di più.

Joint vs Naive La distribuzione delle immagini che ci interessa nel esempio mnist consiste in uno spazio da 784 dimensioni. Dato che questo è irrealizzabile utilizziamo l'approccio naive che assemu che tutte le features sono indipendenti, riducendo il problema a



$$P(X_j \mid Y = y_i) \quad j = 1, \ldots, n$$

1.7.2 Limitazioni

- In alcuni casi in MLE possiamo avere $P(X_i \mid Y) = 0$
- L'approccio naive assume che due eveni sono indipendenti tra loro, e se fossero dipendenti?

1.8 Maximum Likelihood Estimation (MLE)

funzione $P(\textbf{dato}|\theta)$

Ricorda: stima di massima verosimiglianza ($\hat{\theta} = \arg\max_\theta P(dato \mid \theta)$). La frequenza delle classi conta, ovvero una classe più frequente pesa di più in output.

$$\pi_k = P(Y = y_k) = \frac{\#\mathcal{D}\{Y = y_k\}}{|\mathcal{D}|}$$

$$\theta_{ijk} = P(X_i = x_{ij} \mid Y = y_k) = \frac{\#\mathcal{D}\{X_i = x_{ij} \wedge Y = y_k\}}{\#\mathcal{D}\{Y = y_k\}}$$

Perchè? Supponiamo di lanciare una moneta (truccata) 10 volte, otteniamo 6 testa (H) e 4 code (T), possiamo naturalmente concludere che

$$P(H) = .6$$
$$P(T) = .4$$

Supponiamo ora di avere due possibilità:

1. la moneta è reale, quindi $P(H) = .5$ e $P(T) = .5$
2. la moneta è false, quindi $P(H) = .7$ e $P(T) = .3$

Basandoci sulle nostre osservazioni, quale è il nostro caso?

La distribuzione di **Bernoulli** abbiamo 2 possibili output 0 o 1, con probabilità θ e $1 - \theta$. Sia X^n il numero di 0 in una sequenza di n lanci: X^n segue una distribuzione binomiale

$$\begin{aligned} P(X^n = \alpha_0 \mid \theta) &= \binom{n}{\alpha_0} \cdot \theta^{\alpha_0} \cdot (1-\theta)^{\alpha_1} \\ &= \frac{n!}{\alpha_0!(n-\alpha_0)!} \cdot \theta^{\alpha_0} \cdot (1-\theta)^{\alpha_1} \end{aligned}$$

dove $\alpha_1 = n - \alpha_0$ è il numero di 1 nella sequenza (ovvero $\alpha_0 + \alpha_1 = n$).

$$= \frac{n!}{\alpha_0!(n-\alpha_0)!}\theta^{\alpha_0}(1-\theta)^{n-\alpha_0}$$

Continiamo l'esempio di prima $n = 10$, $\alpha_0 = 6$ e $\alpha_1 = 4$.
Se $\theta = .5$ e $1 - \theta = .5$

$$P(X^n = 6 \mid \theta) = \binom{n}{\alpha_0} \cdot \theta^{\alpha_0} \cdot (1-\theta)^{\alpha_1} = \binom{10}{6} \cdot \left(\frac{1}{2}\right)^6 \cdot \left(\frac{1}{2}\right)^4 = .205$$

Se $\theta = .7$ e $1 - \theta = .3$

$$P(X^n = 6 \mid \theta) = \binom{n}{\alpha_0} \cdot \theta^{\alpha_0} \cdot (1-\theta)^{\alpha_1} = \binom{10}{6} \cdot \left(\frac{7}{10}\right)^6 \cdot \left(\frac{3}{10}\right)^4 = .200$$

Quindi $\theta = .5$ è leggermente più verosimile di $\theta = .7$.

Qual è il valore più verosimile per θ?

Calcoliamo MLE (*ricerca max. likelihood*)

$$\hat{\theta} = \arg\max_{\theta} \; P(X^n = \alpha_0 \mid \theta) =$$
$$= \arg\max_{\theta} \; \theta^{\alpha_0} \cdot (1-\theta)^{\alpha_1}$$

In modo equivalente possiamo cercare il θ che massimizza il logaritmo dell'espressione precedente (log-likelihood)

$$\ln\left(\theta^{\alpha_0} \cdot (1-\theta)^{\alpha_1}\right) = \alpha_0 \ln(\theta) + \alpha_1 \ln(1-\theta)$$

Derivando (cerchiamo max) rispetto a θ otteniamo

$$\frac{\alpha_0}{\theta} - \frac{\alpha_1}{1-\theta} = \frac{\alpha_0 - \alpha_0\theta - \alpha_1\theta}{\theta \cdot (1-\theta)}$$

che si annulla per

$$\theta = \frac{\alpha_0}{\alpha_0 + \alpha_1} = \frac{\alpha_0}{n}$$

Caso mulivariabile **Distribuzione discreta** (es. lancio di un dado): k possibili esiti $\{1, 2, \ldots, k\}$ con probabilità θ_i dove $\sum_i \theta_i = 1$.
Sequenza di n lanci: distribuzione multinomiale

$$P(X^n = \alpha_i \mid \theta) = c_{\alpha_i} \prod_i \theta_i^{\alpha_i}$$

dove α_i è il numero di i nella sequenza e $c_{\overline{\alpha_i}}$ è una costante combinatoria che non dipende da θ.

$$\textbf{MLE}: \quad \theta_i = \frac{\alpha_i}{\sum_i \alpha_i} = \frac{\alpha_i}{n}$$

1.9 Classificazione documento

L'evento X_i = i-esima parola nel documento: una **variabile aleatoria discreta** che assume tanti valori quante sono le parole del linguaggio.

$$\theta_{i,word,\ell} = P(X_i = word \mid Y = \ell)$$

è la probabilità che, in un documento della categoria "ℓ", la parola "$word$" appaia nella posizione "i".

Assumiamo che tutti gli eventi siano indipendenti e che abbiano una distribuzione indipendente dalla posizione:

$$\theta_{i,word,\ell} = \theta_{j,word,\ell} = \theta_{word,\ell}$$

Sono le ipotesi del Naïve Bayes multinomiale: entrambe sono semplificazioni (le parole non sono davvero indipendenti e la loro probabilità dipende dalla posizione), ma servono a ridurre i parametri a uno per parola e per classe.

Training e classificazione Variabili aleatorie discrete X_i, Y.

Training

- per ogni possibile valore y_k di Y stimiamo

$$\pi_k = P(Y = y_k)$$

- per ogni possibile valore x_{ij} dell'attributo X_i stimiamo

$$\theta_{ijk} = P(X_i = x_{ij} \mid Y = y_k)$$

Classificazione di $a^{new} = \langle a_1, \ldots, a_n \rangle$ (sequenza di parole)

$$\begin{aligned} Y^{new} &= \arg\max_{y_k} \; P(Y = y_k) \cdot \prod_i P(X_i = a_i \mid Y = y_k) \\ &= \arg\max_{k} \; \pi_k \cdot \prod_i \theta_{ijk} \end{aligned}$$

dove $x_{ij} = a_i$.

1.9.1 Maximum likelihood estimates (MLE's)

- $\pi_k = P(Y = y_k)$
  frazione dei documenti nella categoria y_k
- $\theta_{i,word,k} = \theta_{word,k} = P(X = word \mid Y = y_k)$
  frequenza della parola "*word*" nei documenti della categoria y_k

1.9.2 Log likelihood

Prima abbiamo parlato di classificazione $a^{new} = \langle a_1, \ldots, a_n \rangle$ come

$$\begin{aligned} Y^{new} &= \arg\max_{y_k} \; P(Y = y_k) \cdot \prod_i P(X_i = w_j \mid Y = y_k) \\ &= \arg\max_{k} \; \pi_k \cdot \prod_i \theta_{ijk} \end{aligned}$$

Però possiamo calcolarlo in `log`, che ci da qualche vantaggio numerico (arrotondiamo meno su `double` o `float`) e matematico grazie alle proprietà dei logatirmi $log(a \cdot b) = log(a) + log(b)$:

$$\begin{aligned} Y^{new} &= \arg\max_{y_k} \; \log\left(P(Y = y_k) \cdot \prod_i P(X_i = w_j \mid Y = y_k) \right) \\ &= \arg\max_{k} \; \log(\pi_k) + \sum_i \log(\theta_{ijk}) \end{aligned}$$

Inoltre, se $\theta_{ijk} = \theta_{i'jk} = \theta_{jk}$

$$\underbrace{\sum_i \log(\theta_{ijk})}_{\text{sulle } \textbf{posizioni } i} = \underbrace{\sum_j n_j \cdot \log(\theta_{jk})}_{\text{sulle } \textbf{parole } j}$$

dove n_j è il numero di occorrenze della parola w_j nel documento da classificare.
La formula dice che puoi sommare per posizione oppure per parola distinta, e il risultato è lo stesso.

1.9.3 Prodotto scalare, correlation, Cosine similarity

Consideriamo vettori della dimensione del vocabolario, cioè ogni parola è una dimensione diversa.

Training
Per ogni categoria k costruiamo un vettore *"spettrale"* (non centra con il teorema spettrale. Il vettore spettrale di una categoria è la "carta d'identità" di quella categoria. Per ogni parola dice quanto quella parola è tipica della categoria.)

$$\vec{s}_k = \langle \log(\theta_{jk}) \rangle_{j \in words}$$

dove θ_{jk} = frequenza della parola j nei documenti della categoria k.

Dato un nuovo documento, calcoliamo il vettore *"dei conteggi"* (Il vettore dei conteggi di un documento è la sua "lista della spesa". Per ogni parola dice quante volte compare nel documento, 0 se è assente.)

$$\vec{d} = \langle n_j \rangle_{j \in words}$$

e classifichiamo il documento come

$$\arg\max_k \underbrace{\vec{d} \cdot \vec{s}_k}_{\text{prodotto scalare}} = \sum_j d_j \cdot s_{jk}$$

Prodotto scalare, geometricamente e analiticamente

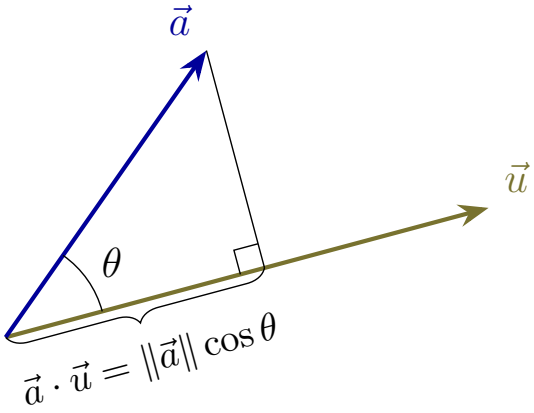
Definizione geometrica

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos(\theta)$$

dove θ è l'angolo tra i due vettori.

Con $\|\vec{a}\|$ indichiamo la **norma euclidea** (la lunghezza) del vettore

$$\|\vec{a}\| = \sqrt{\sum_i a_i^2} = \sqrt{\vec{a} \cdot \vec{a}}$$



Definizione analitica Dati $\vec{a} = (a_1, a_2, \ldots, a_n)$ e $\vec{b} = (b_1, b_2, \ldots, b_n)$

$$\vec{a} \cdot \vec{b} = \sum_{i=1}^{n} a_i b_i$$

La cosine similarity tra $\vec{a}$ e $\vec{b}$ è il prodotto scalare normalizzato rispetto alla lunghezza (magnitudine) dei due vettori:

$$S_C(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|} = \cos(\theta)$$

Dimostrazione: dal teorema del coseno si ricava $\vec{a} \cdot \vec{b} = \frac{\|\vec{a}\|^2 + \|\vec{b}\|^2 - \|\vec{a} - \vec{b}\|^2}{2}$; sviluppando i quadrati i termini a_i^2 e b_i^2 si cancellano e resta $\sum_i a_i b_i$, quindi le due definizioni coincidono.

1.10 La natura lineare di Naïve Bayes (caso booleano)

Ricorda: assumiamo che le features sono indipendenti.
X_i, Y booleane

Classificazione di $\vec{x} = \langle x_1, \ldots, x_n \rangle$ (vediamo se rientra più in $Y = 0$ oppure $Y = 1$):

$$\frac{P(Y = 1 \mid X_1 \ldots X_n = \vec{x})}{P(Y = 0 \mid X_1 \ldots X_n = \vec{x})} = \frac{P(Y = 1) \prod_i P(X_i = x_i \mid Y = 1)}{P(Y = 0) \prod_i P(X_i = x_i \mid Y = 0)} \geq 1$$

Passando ai logaritmi (per vantaggi matematici come prima)

$$\log \frac{P(Y = 1)}{P(Y = 0)} + \sum_i \log \frac{P(X_i = x_i \mid Y = 1)}{P(X_i = x_i \mid Y = 0)} \geq 0$$

Usiamo il fatto che per una funzione booleana $f(x)$ vale

$$f(x) = x \cdot f(1) + (1 - x) \cdot f(0)$$

Quindi, se $\theta_{ik} = P(X_i = 1 \mid y = k)$ (di conseguenza $P(X_i = 0 \mid y = k) = 1 - \theta_{ik}$), otteniamo

$$\underbrace{\log \frac{P(Y = 1)}{P(Y = 0)} + \sum_i x_i \cdot \log \frac{\theta_{i1}}{\theta_{i0}} + \sum_i (1 - x_i) \cdot \log \frac{1 - \theta_{i1}}{1 - \theta_{i0}} \geq 0}_{\text{i pesi } \vec{w} \text{ sono gia inclusi nei logaritmi}}$$

che è un'espressione lineare nell'insieme delle features x_i!

1.10.1 Classificazione lineare

Algoritmi di classificazione basati su una combinazione lineare dei valori delle features.

Ogni feature è valutata indipendentemente dalle altre e contribuisce al risultato in modo lineare, con un opportuno peso (che è un parametro del modello, da stimare).

Ad esempio, se le features sono i pixel di un'immagine, possiamo usare metodi lineari solo a meno di una normalizzazione (in posizione e dimensione) dell'oggetto da riconoscere.

1.11 Gaussian Naïve Bayes

Un limite del Naïve Bayes è che le features X_i non devono essere continue, perché se lo fossero le probabilità $P(X_i \mid Y)$ sono nulle. Quindi parliamo di **densità** di distribuzioni.

Un approccio è quello di supporre che $P(X_i \mid Y)$ abbia una distribuzione Gaussiana (detta anche Normale).

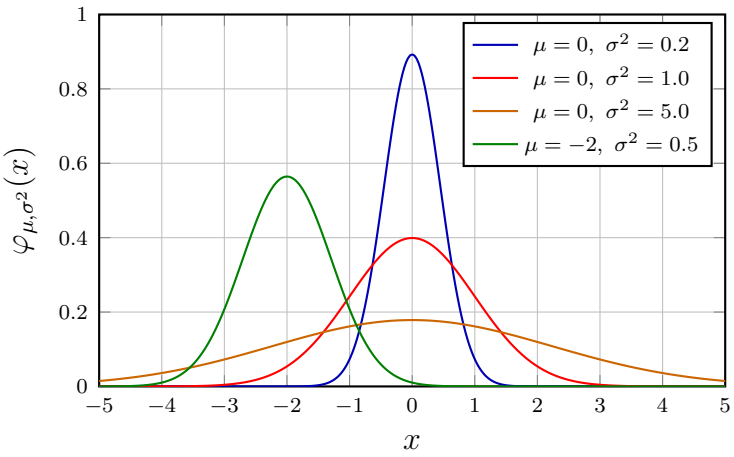
Densità di probabilità (con integrale = 1):

$$p(x \mid \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

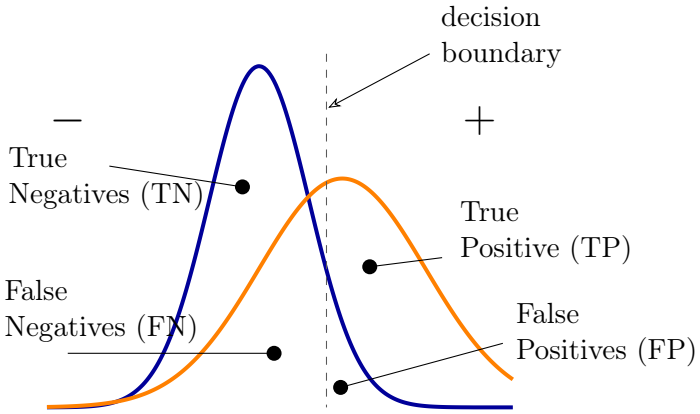
valore medio
$E[X] = \mu$

varianza
$Var[X] = \sigma^2$

deviazione standard
$\sigma_X = \sigma$



1.12 Accuratezza, Precisione e Richiamo



$$\text{Accuratezza} = \frac{TP + TN}{All}$$ Quante istanze sono classificate correttamente?

$$\text{Precisione} = \frac{TP}{TP + FP}$$ Quanto è precisa la classificazione (sui positivi)?

$$\text{Richiamo} = \frac{TP}{TP + FN}$$ Che percentuale dei positivi è recuperata?

$$F1 = 2\frac{\text{Precisione} \cdot \text{Richiamo}}{\text{Precisione} + \text{Richiamo}}$$ Media armonica tra precisione e richiamo.

1.12.1 Parametri descrittivi del modello

Assumiamo che per ogni valore y_k di Y la variabile aleatoria $P(X_i \mid Y = y_k)$ abbia una distribuzione Gaussiana:

$$N(x \mid \mu_{ik}, \sigma_{ik}) = \frac{1}{\sigma_{ik}\sqrt{2\pi}} e^{-\frac{(x-\mu_{ik})^2}{2\sigma_{ik}^2}}$$

Apprendimento: stimare i valori dei parametri μ_{ik}, σ_{ik} e $\pi_k = P(Y = y_k)$.

Classificazione di $x^{new} = \langle a_1, \ldots, a_n \rangle$:

$$\begin{aligned} Y^{new} &= \arg\max_{y_k} P(Y = y_k) \cdot \prod_i P(X_i = a_i \mid Y = y_k) \\ &= \arg\max_k \pi_k \cdot \prod_i N(a_i \mid \mu_{ik}, \sigma_{ik}) \end{aligned}$$

1.12.2 Maximum Likelihood Estimates

μ_{ik} = **valore medio** di X_i per i dati con etichetta $Y = y_k$. Formalmente:

$$\mu_{ik} = \frac{\sum_j X_i^j \, \delta(Y^j = y_k)}{\sum_j \delta(Y^j = y_k)}$$

dove j varia sulle istanze del training set e

$$\delta(Y^j = y_k) = \begin{cases} 1 & \text{se } Y^j = y_k \\ 0 & \text{altrimenti} \end{cases}$$

σ_{ik}^2 = **varianza** di X_i per le istanze con etichetta $Y = y_k$:

$$\sigma_{ik}^2 = \frac{\sum_j (X_i^j - \mu_{ik})^2 \, \delta(Y^j = y_k)}{\sum_j \delta(Y^j = y_k)}$$

1.13 Classificatori discriminativi

1.13.1 Regressione Logistica

Ricorda: non assumiammo neessuna forma di dipendenza/indpendeza.
Idea:

- Naïve Bayes ci permette di calcolare $P(Y \mid X)$ dopo aver appreso $P(Y)$ e $P(X \mid Y)$.
- Perché non cercare di apprendere direttamente $P(Y \mid X)$?

Che forma adottare per $P(Y \mid X)$? Che forma abbiamo nel caso di Naïve Bayes? Se:

- Y è una variabile aleatoria booleana;
- X_i sono variabili aleatorie continue;
- gli X_i sono indipendenti l'uno dall'altro data Y;
- $P(X_i \mid Y = k)$ hanno distribuzioni Gaussiane $N(\mu_{ik}, \sigma_i)$ (attenzione: non σ_{ik}!);
- Y ha una distribuzione di Bernoulli (π).

Allora:

$$P(Y = 1 \mid X = \langle x_1, \ldots, x_n \rangle) = \frac{1}{1 + \exp\left(w_0 + \sum_i w_i x_i\right)}$$

Dimostrazione: omessa.

Una proprietà interessante è

$$\ln \frac{P(Y = 0 \mid X = \langle x_1, \ldots, x_n \rangle)}{P(Y = 1 \mid X = \langle x_1, \ldots, x_n \rangle)} = w_0 + \sum_i w_i x_i$$

Per classificare $X = \langle x_1, \ldots, x_n \rangle$ è sufficiente vedere se $w_0 + \sum_i w_i x_i > 0$.

1.13.2 La funzione logistica

La Regressione Logistica *assume* che

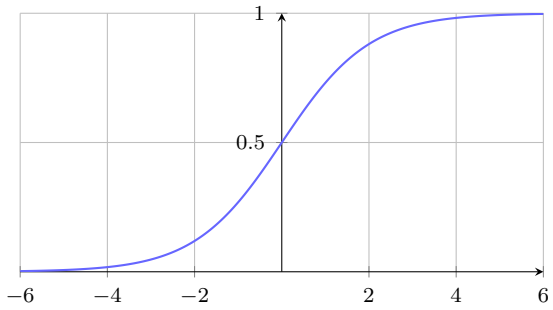
$$P(Y = 1 \mid X = \langle x_1, \ldots, x_n \rangle) = \frac{1}{1 + \exp\left(-w_0 - \sum_i w_i x_i\right)}$$

e cerca direttamente di stimare i parametri w_i.

La funzione

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

è detta funzione logistica.



1.13.3 Training per regressione logistica (caso binario)

Sappiamo che

$$P(y = 1 \mid x, w) = \sigma\left(w_0 + \sum_i w_i x_i\right)$$

Dati i campioni (indipendenti) $\langle x^\ell, y^\ell \rangle$, la loro probabilità è

$$\prod_\ell P(y^\ell \mid x^\ell, w) = \prod_\ell P(y^\ell = 1 \mid x^\ell, w)^{y^\ell} \cdot P(y^\ell = 0 \mid x^\ell, w)^{(1-y^\ell)}$$

Vogliamo trovare i valori dei parametri $\vec{w}$ che massimizzano questa probabilità (MLE, ovvero likelihood).

Analogamente, possiamo passare ai logaritmi e massimizzare

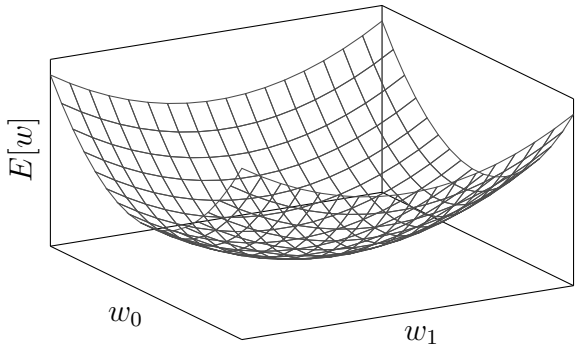
$$\mathcal{L}(\vec{w}) = \sum_\ell \log P(y^\ell \mid x^\ell, w) = \sum_\ell \left(y^\ell \cdot \log P(Y = 1 \mid x^\ell, w) + (1 - y^\ell) \cdot \log P(Y = 0 \mid x^\ell, w)\right)$$

1.13.4 Gradiente

Sfortunatamente non esiste una soluzione analitica per il precedente problema di ottimizzazione, ovvero:

$$\boxed{\mathbf{w}^* = \arg\max_{\mathbf{w}} \mathcal{L}(\mathbf{w})}$$

Quindi, utilizziamo dei metodi iterativi di ottimizzazione, come la tecnica del gradiente:



$$\nabla_w[E] = \left[\frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_n}\right]$$

training:

$$\Delta w_i = \underbrace{\mu}_{\text{learning rate}} \cdot \frac{\partial E}{\partial w_i}$$

passo del gradiente

$$w_i = w_i + \Delta w_i$$

Perché il segno è +? Qui E è la log-likelihood, da **massimizzare**. Il gradiente punta nella direzione in cui la funzione **cresce**, quindi per salire ci si muove **nella** direzione del gradiente: ascesa del gradiente $\Rightarrow \Delta w_i = +\mu \cdot \frac{\partial E}{\partial w_i}$. a

1.14 La tecnica del gradiente

1.14.1 Approccio iterativo

L'obiettivo consiste nel minimizzare una qualche funzione di errore $\Theta(w)$ misurata sul training set, aggiustando opportunamente i parametri del modello.

Possiamo raggiungere un minimo per $\Theta(w)$ compiendo iterativamente dei piccoli passi nella direzione opposta al gradiente (discesa del gradiente).

Si tratta di una **tecnica generale**, tuttavia non garantisce di raggiungere minimi globali.

Passo e learning rate Il passo è lo spostamento dei parametri a ogni iterazione:

$$\Delta w_i = -\mu \cdot \frac{\partial \Theta}{\partial w_i} \qquad\qquad w_i \leftarrow w_i + \Delta w_i$$

- il **gradiente** dà la **direzione** (si va nel verso opposto, perché si minimizza);
- il learning rate μ (spesso indicato con η) è lo scalare che ne decide l'**ampiezza**: passo $= \mu \cdot$ gradiente.

Perché qui il segno è $-$? Qui Θ è una funzione di errore, da **minimizzare**: per scendere ci si muove **contro** il gradiente: discesa del gradiente $\Rightarrow \Delta w_i = -\mu \cdot \frac{\partial \Theta}{\partial w_i}$.

Nella sezione precedente invece compare $+$ perché lì E è la log-likelihood, da **massimizzare** (ascesa). Le due cose **coincidono**: massimizzare la log-likelihood equivale a minimizzare il suo opposto $\Theta = -E$ (*cross-entropy*), e

$$-\mu \cdot \frac{\partial \Theta}{\partial w_i} = -\mu \cdot \frac{\partial(-E)}{\partial w_i} = +\mu \cdot \frac{\partial E}{\partial w_i}$$

Regola: errore/loss (minimizzo) $\Rightarrow -$; likelihood/reward (massimizzo) $\Rightarrow +$.

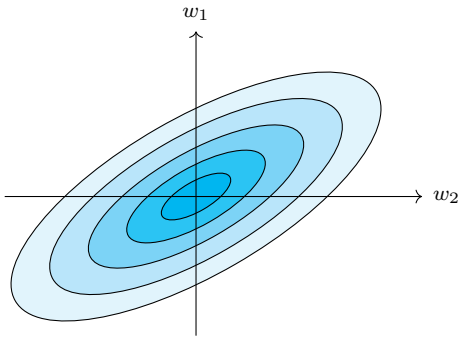
Il learning rate è un **iperparametro**: non viene appreso dal training, si sceglie a mano (non è una metrica).

- μ troppo alto: training più veloce, ma i passi possono **saltare sopra al minimo**, oscillare o divergere;
- μ troppo basso: convergenza molto lenta, e più facile restare bloccati in minimi locali;
- può **variare durante il training** (*learning rate schedule / decay*): conviene partire alto e **decrementarlo verso la fine**, per avvicinarsi al minimo con passi più fini.

1.14.2 La superficie di errore

Immaginate di avere un asse "orizzontale" per ogni parametro e un asse verticale per l'errore.

Il caso ideale si ha quando la superficie è convessa, cosa che garantisce l'esistenza di un unico minimo globale.

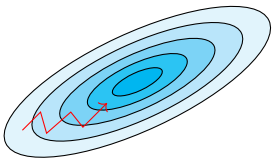


La superficie di errore nel caso della regressione logistica (con log-likelihood) è convessa.

Per deep neural networks la superficie di errore può essere molto più complicata (molti minimi locali).

1.14.3 Online vs Batch learning

Su quanti punti misurare l'errore.



Full batch: si calcola l'errore e il suo gradiente sull'intero training set. Il gradiente punta nella direzione di massima ascesa, perpendicolare alle linee di livello.

Online: si calcola l'errore e il suo gradiente su una istanza del training set alla volta. Veloce ma impreciso: il gradiente induce un cammino a zig-zag attorno alla direzione corretta.

Mini-batch: calcolo dell'errore e del suo gradiente su un sottoinsieme dei dati: un buon compromesso.

1.14.4 Gradiente per regressione logistica

- probabilità che una istanza ℓ abbia etichetta $Y = 1$:

$$\alpha^\ell = P(Y = 1 \mid x^\ell, w) = \sigma\left(w_0 + \sum_i w_i x_i^\ell\right)$$

- Log-likelihood:

$$l(w) = \sum_\ell \log P(Y = y^\ell \mid x^\ell, w) = \sum_\ell y^\ell \log(\alpha^\ell) + (1 - y^\ell)\log(1 - \alpha^\ell)$$

- gradiente (*dimostrazione omessa*):

$$\frac{\partial l(w)}{\partial w_i} = \sum_\ell x_i^\ell \cdot (y^\ell - \alpha^\ell)$$

1.14.5 Il processo di apprendimento

Iterare la seguente operazione di update finché non si raggiunge l'approssimazione desiderata (ad esempio fino a quando l'accuratezza sul set di validazione è soddisfacente o non sembra aumentare).

$$\begin{aligned}
w_i &\leftarrow w_i + \Delta w_i \\
w_i &\leftarrow w_i + \mu \frac{\partial l(w)}{\partial w_i} \\
w_i &\leftarrow w_i + \mu \sum_\ell x_i^\ell \cdot \left(y^\ell - \alpha^\ell\right) \\
w_i &\leftarrow w_i + \mu \sum_\ell x_i^\ell \cdot \left(y^\ell - P(Y = 1 \mid x^\ell, w)\right)
\end{aligned}$$

Frequentemente si aggiunge un qualche regolarizzatore:

$$w_i \leftarrow w_i - \mu\lambda w_i + \mu \sum_\ell x_i^\ell \cdot \left(y^\ell - P(Y = 1 \mid x^\ell, w)\right)$$

- cerca di tenere i parametri w_i vicini a 0;
- tende a ridurre l'overfitting.

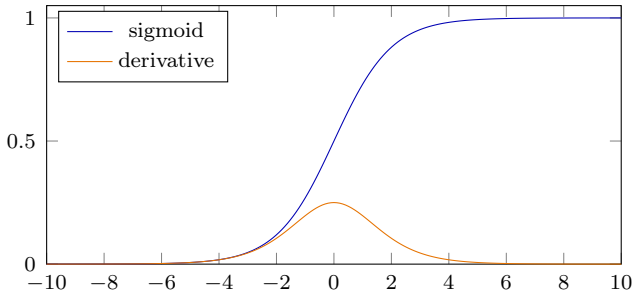
1.14.6 Osservazione sulla derivata della sigmoid

Nella slide precedente abbiamo visto che

$$\frac{\partial \log(\alpha^\ell)}{\partial z} = \frac{1}{\sigma(z)} \frac{\partial \sigma(z)}{\partial z} = 1 - \sigma(z)$$

Dunque:

$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z)\,(1 - \sigma(z))$$



La derivata della funzione logistica è molto piatta: vale al massimo $\frac{1}{4}$ (in $z = 0$) e tende rapidamente a 0 al crescere di $|z|$. Quando z è lontano da 0 la sigmoid è **satura** (output vicino a 0 o a 1): modificare i pesi non cambia quasi l'output, quindi il segnale di correzione è molto debole.

- Nella regressione logistica questo non è un problema: nel gradiente $\sum_\ell x_i^\ell (y^\ell - \alpha^\ell)$ il fattore $\sigma'(z)$ non compare, perché si semplifica con l'$\frac{1}{\alpha^\ell}$ prodotto dal logaritmo.
- Nelle reti profonde invece il gradiente attraversa molte sigmoid, e a ogni passaggio viene moltiplicato per un fattore $\leq \frac{1}{4}$: dopo pochi layer è quasi nullo (*vanishing gradient*).

1.15 Modelli generativi e discriminativi

1.15.1 Generative vs discriminative models

Discriminativi: i modelli discriminativi sono utilizzati prevalentemente per problemi di classificazione, dove l'obiettivo consiste nell'apprendere la frontiera (decision boundary) che separa le differenti classi.

Generativi: i modelli generativi cercano di apprendere la distribuzione dei dati. Oltre che ai fini della classificazione, possono essere usati per data generation, data augmentation (con cautela), anomaly detection, trasferimento di contenuto/stile e molte altre applicazioni.

1.15.2 Modelli generativi e discriminativi (esempio)

Problema: stimare $f : X \to Y$ o $P(Y \mid X)$.

Classificatore generativo (e.g., Naïve Bayes):

- si assume una certa distribuzione per $P(X \mid Y)$, $P(X)$;
- si stimano i parametri per $P(X \mid Y)$ e $P(X)$ sui dati di training;
- si utilizza la regola di Bayes per inferire $P(Y \mid X)$.

Classificatore discriminativo (e.g., Regressione Logistica):

- si assume una certa distribuzione per $P(Y \mid X)$;
- si stimano i parametri per $P(Y \mid X)$ sui dati di training.

1.16 Esempio: classificazione di cifre MNIST con Regressione Logistica

1.16.1 I metodi lineari sono potenti

Si potrebbe avere l'impressione errata che i metodi lineari non siano di interesse effettivo in quanto troppo deboli.

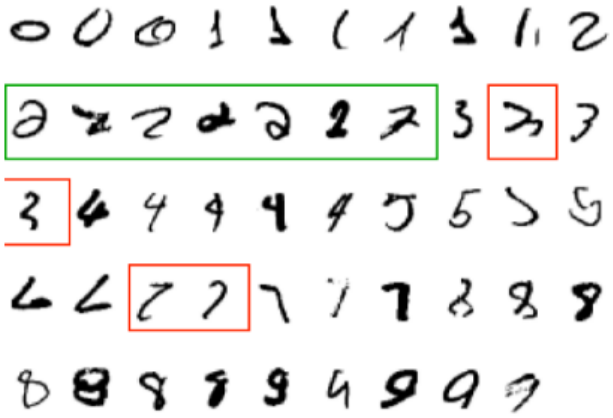
Quando si hanno molte features a disposizione e i dati sono stati adeguatamente pre-processati anche i metodi lineari funzionano bene.

In uno spazio con molte dimensioni i dati assumono una distribuzione molto sparsa, quindi diventa plausibile discriminare le varie classi con dei semplici iperpiani.

1.16.2 Il dataset MNIST

Modified **N**ational **I**nstitute of **S**tandards and **T**echnology database

- immagini in scala di grigio di cifre numeriche scritte a mano. Bassa risoluzione: 28×28 pixels;
- 60,000 immagini di training e 10,000 immagini di testing.



Nel caso delle immagini ogni pixel è una feature. Una piccola immagine 28×28 contiene già 784 features.

1.17 Classificazione con regressione multinomiale

1.17.1 Classificazione con regressione multinomiale

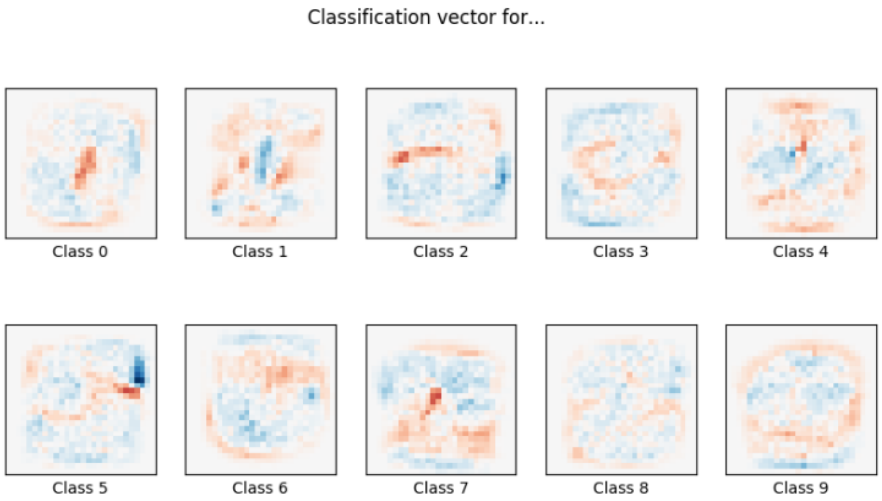
DEMO!

```
Sparsity with L1 penalty: 25.18%
Test score with L1 penalty: 0.9257
Example run in 22.404 s
```

1.17.2 Visualizzazione dei pesi

Per ogni classe, i pesi corrispondenti hanno la stessa dimensionalità dell'immagine di input: abbiamo un peso per ogni feature.

Possiamo visualizzare i pesi!

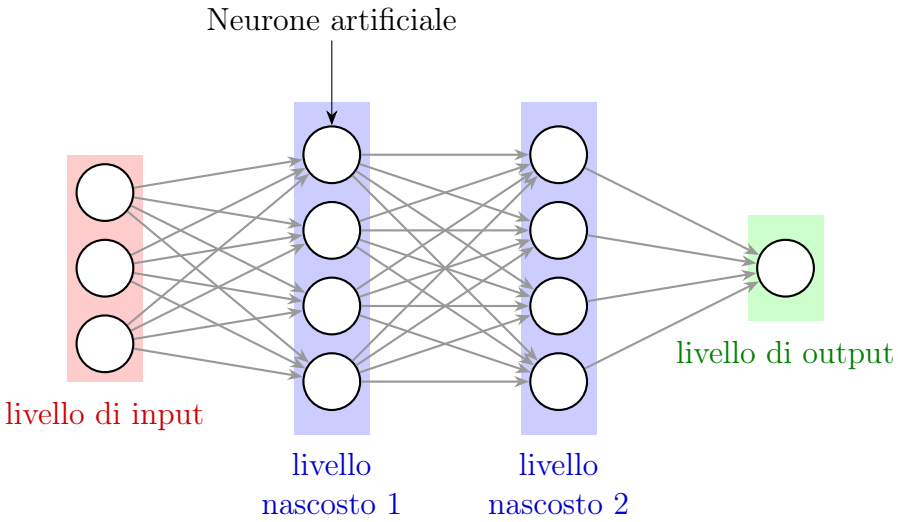


Per ogni classe, le zone rosse sono quelle dove mi aspetto di avere della scrittura, quelle blu dove non mi aspetto di averne.

2 Introduzione alle reti neurali

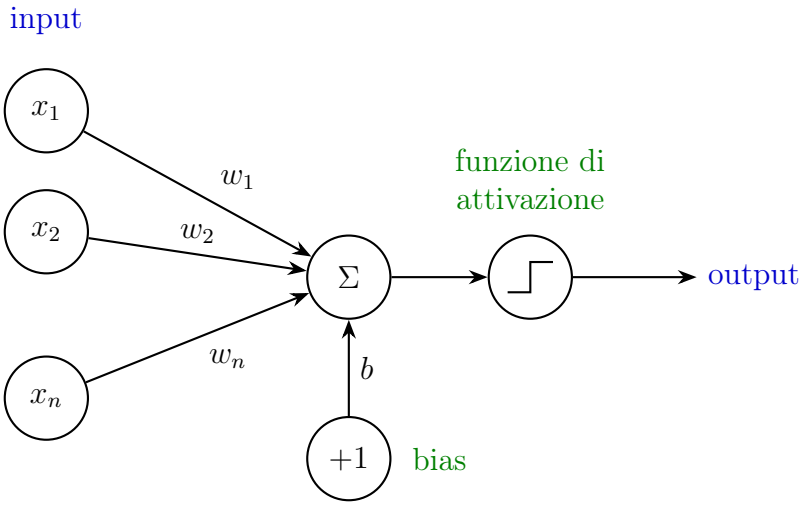
- **Cos'è il deep learning?**
  Un ramo del machine learning.
- **Quali problemi può risolvere?**
  Gli stessi del ML.
- **Ci sono casi in cui il DL eccelle particolarmente?**
  Tutti i problemi con miriadi di features (immagini, audio, testo, ...).

Quale è la tecnica base del DL? Le *reti neurali!* Il nome `deep` deriva dal fatto che abbiamo una rete di neuroni artificiali molto profonda. Inoltre possiamo dire che "*sfrutta*" la profondità delle features, ovvero sfrutta caratteristiche profonde dei dati, caratteristiche estratte da altre caratteristiche.



Ogni neurone riceve più input e produce un singolo output (che può essere passato come input a molti altri neuroni).

2.1 Il neurone artificiale

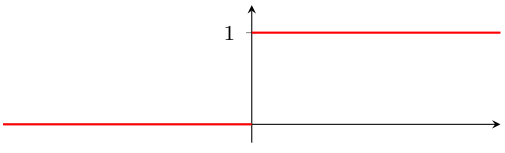


Ogni neurone (!) implementa un regressore logistico

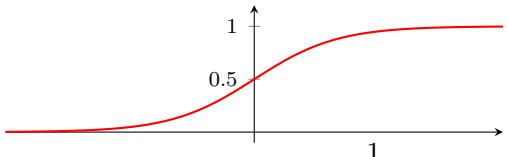
$$\sigma(wx + b)$$

2.2 Funzioni di attivazione

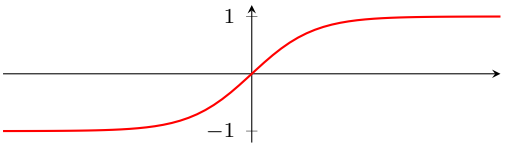
La funzione di attivazione è la responsabile dello scatto (*threshold triggering*) del neurone.



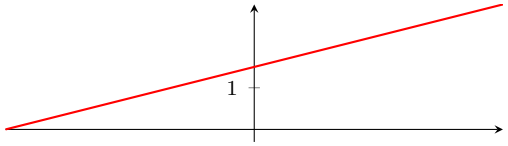
soglia: se $x > 0$ allora 1 altrimenti 0



funzione logistica: $\dfrac{1}{1+e^{-x}}$



tangente iperbolica: $\dfrac{e^x - e^{-x}}{e^x + e^{-x}}$



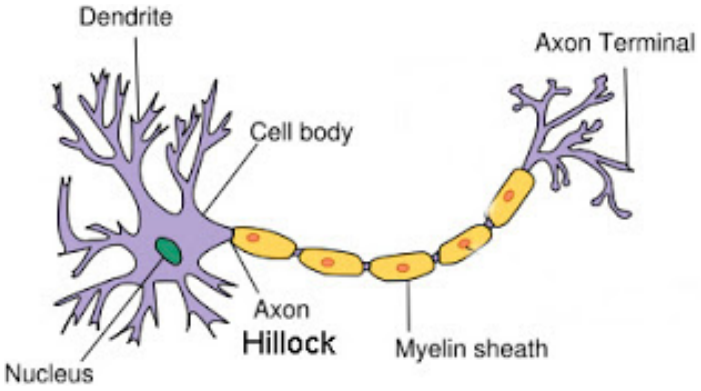
rectified linear (ReLU): se $x > 0$ allora x altrimenti 0

2.3 Il neurone corticale

Il neurone biologico funziona in modo analogo:

- l'albero dendritico della cellula raccoglie gli input provenienti dagli altri neuroni, che vengono sommati tra loro;
- quando viene superata una soglia di innesco, l'*Axon Hillock* (cono di emergenza dell'assone) genera un impulso che viene trasmesso lungo l'assone agli altri neuroni.

La struttura della cellula comprende: dendriti, corpo cellulare, nucleo, *axon hillock*, guaina mielinica e terminali dell'assone.



2.3.1 Alcune cifre sul cervello umano

- numero di neuroni: $\sim 2 \cdot 10^{10}$;
- tempo di commutazione di un neurone: ~ 0.001 s (lento!);
- sinapsi (connessioni) per neurone: $\sim 10^{4-5}$;
- tempo per riconoscere un'immagine: ~ 0.1 s.

Quindi il cervello è poco profondo (< 100 livelli) ma ha un altissimo parallelismo.

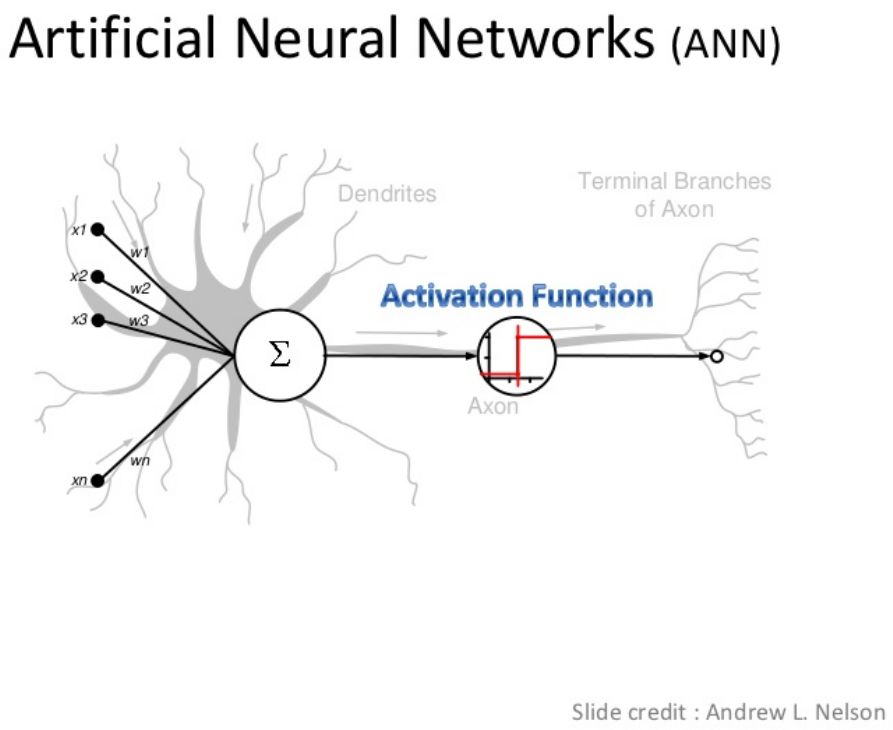


2.3.2 Confronto con il neurone corticale

Le reti neurali artificiali (ANN) ricalcano il neurone biologico:

- gli input $x_1, \ldots, x_n$ con i pesi $w_1, \ldots, w_n$ corrispondono ai *dendriti*;
- il Σ somma gli input pesati (corpo cellulare);

- la *funzione di attivazione* fa da soglia, come l'*assone*;
- l'output viene propagato ai rami terminali dell'assone, cioè agli altri neuroni.



2.3.3 Motivazioni del calcolo neurale

- capire, tramite simulazione, come funziona il cervello;
- investigare un diverso paradigma di computazione, molto lontano dai linguaggi di programmazione tradizionali;
- risolvere problemi pratici difficili da affrontare con tecniche algoritmiche, utile anche se il cervello funziona in modo diverso.

2.4 Topologie di rete

2.4.1 Reti feed-forward e ricorrenti

Se la rete è aciclica si chiama *feed-forward*.
Se ha dei cicli si chiama *ricorrente* (non verranno trattate).

2.4.2 Livelli (layers)

In una rete feed-forward i neuroni sono di solito organizzati in livelli (*layers*): input, hidden e output.
Se c'è più di un livello nascosto la rete è profonda (*deep*), altrimenti si dice poco profonda (*shallow*).

2.4.3 Livelli principali: dense layer

Dense layer: ogni neurone del livello $k-1$ è connesso a **ogni** neurone del livello k.

Un singolo neurone:

$$I^n \cdot W^n + B^1 = O^1$$

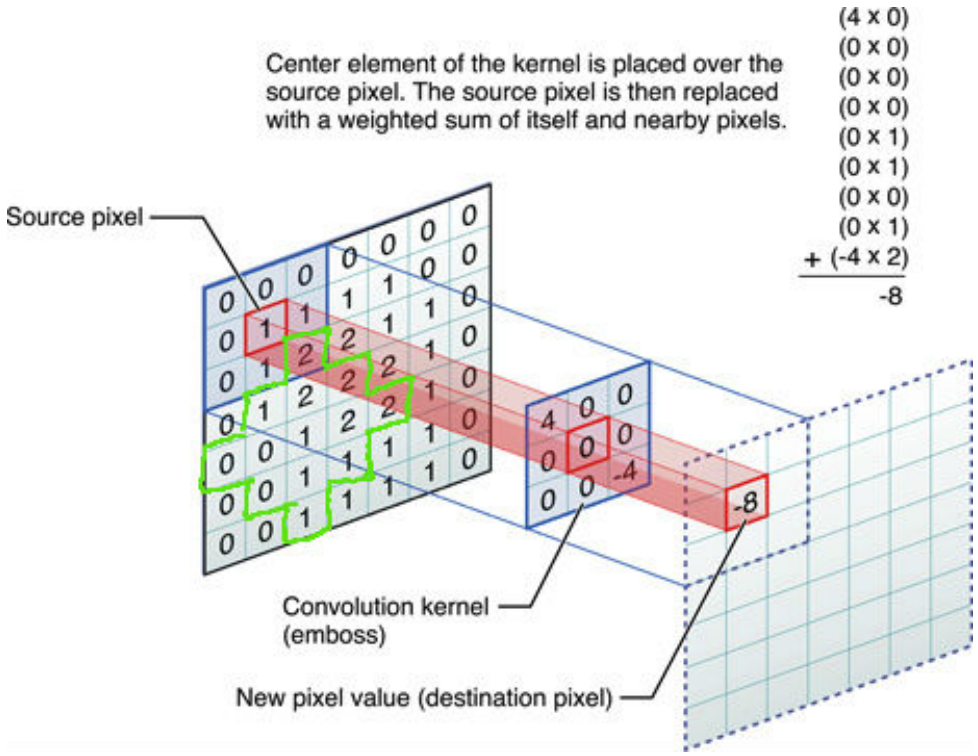
L'operazione può essere vettorizzata per produrre m output in parallelo:

$$I^n \cdot W^{n \times m} + B^m = O^m$$

- i dense layer lavorano di solito su input piatti (non strutturati);
- l'ordine degli elementi in input è irrilevante.

2.4.4 Livelli principali: convolutional layer

Convolutional layer: ogni neurone del livello $k-1$ è connesso, tramite un **kernel** parametrico, a un sottoinsieme fisso di neuroni del livello k. Il kernel viene fatto scorrere (*convoluto*) su tutto il livello precedente.



1. si sposta il kernel K su una porzione M dell'input di uguale dimensione;
2. si calcola il prodotto scalare $M \cdot K$ e, eventualmente, si aggiunge un bias;
3. si sposta il kernel e si ripete.

La dimensione dell'output dipende solo da quante volte viene applicato il kernel.

L'input è strutturato e la struttura si riflette nell'output.

2.5 Approfondimento sul DL

2.5.1 Capire il DL

- capire i **diversi livelli** e il loro scopo;
- capire come i livelli possono essere organizzati in **architetture rilevanti**;
- capire le diverse **applicazioni** possibili del DL e le loro soluzioni specifiche;
- capire i principali **problemi** e **costi**.

2.5.2 Framework per il DL

- TensorFlow/Keras, Google Brain;
- PyTorch, Facebook;
- MXNET, Apache.

Useremo principalmente Keras.

2.5.3 Cenni storici

2.5.4 Il lascito (legacy)

1958	perceptron
1975	backpropagation
1980	convolutional layers
1992	max-pooling
1997	LSTM
...	...

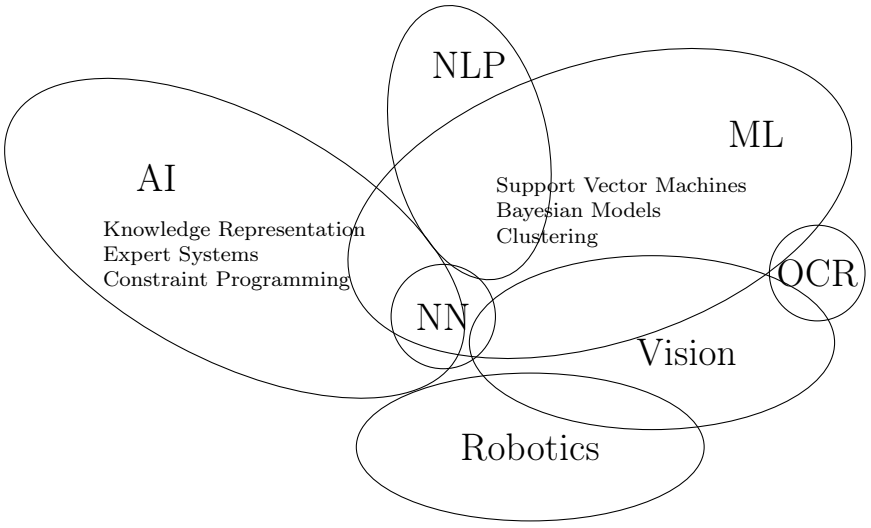
Progresso estremamente lento.

2.5.5 La rivoluzione del Deep Learning

2011	**Google Brain foundation**
2012	ReLU and Dropout
2012	ImageNet Competition
2013	DQN
2014	Inception v1
2015	**TensorFlow release**
2015	Keras release
2015	Batch normalization
2015	YOLO v1
2015	**OpenAI foundation**
2016	Residual connections
2017	**PyTorch release**
2017	Mask-RCNN
2017	PPO
2018	Transformers
2018	BERT
2018	GPT
2018	Soft Actor Critic
2020	OpenAI Jukebox
2020	Vision Transformer
2021	**MXNet release**
2022	DALL-E

Solo alcune pietre miliari...

2.5.6 La situazione a inizio secolo



2.5.7 L'era del deep learning

